Gizmonic Cambot: setup and installation guide

For people and for coding agents (Cursor, Claude Code, Codex, …)

Gizmonic Cambot project · CC BY 4.0
Guide rev A · targets firmware v0.x · October 2026

- Proven on
- 0. Agent contract (read first)
- 1. What you need
- S1. Install the tools
- S2. Insert the SD card
- S3. Connect the TTL port
- S4. Flash the firmware ⚠
- S5. Join Wi-Fi
- S6. Verify the camera ⚠ (SD format if needed)
- S7. Connect a model (optional)
- S8. Describe the scene and what to watch for
- S9. Mount and aim
- S10. Add the camera to your editor (MCP)
- 9. Troubleshooting reference
- Appendix A. Log keys

> **Status.** This guide describes the v1 setup flow planned in the repository plan. Commands under `cambot` ship with milestone M2 and are not in this repository yet. Until then, flash the setup test in `firmware/`. It joins Wi-Fi and registers with the registrar on this computer. Do not invent `cambot` commands.

Proven on

This guide was first carried out on the host documented in profiles/macos-arm64.json. Notes from that run are profiles/macos-arm64.md. Treat that pair as a worked example for Apple Silicon macOS, not as your machine.

An agent reads this file first, then the closest profile for the computer it is on. If these steps fail on another system, or you learn something useful for Linux, Windows, or another macOS layout, add `profiles/<id>.json` matching schema/host-profile.schema.json and `profiles/<id>.md` describing what that computer needed, leave every secret value out of both files, and open a pull request so other agents can reuse the tips.

0. Agent contract (read first)

This section is for coding agents. People can skip to section 1.

Goal: a Gizmonic Cambot is recording to its SD card, reachable on the user's LAN at `http://cambot-XXXX.local/`, and registered as an MCP server in the user's editor.

Ask the user for these before starting. Never guess them:

Input	Notes
Wi-Fi SSID	Must be a **2.4 GHz** network, WPA2 or WPA3 Personal
Wi-Fi password	Have the user type it into a hidden prompt, or set `CAMBOT_WIFI_PASSWORD` themselves
Model endpoint	Base URL, model name, API key. Optional; recording works without it.
Time zone	IANA name such as `America/Los_Angeles`; set `CAMBOT_TZ` or `TZ` on the host
What the camera watches	One or two sentences, for the scene profile

Rules:

1. **Never print, log, echo or commit** the Wi-Fi password, the model API key or the camera token. Pass them through environment variables, stdin (`-`) or a hidden prompt. If a secret appears in output, tell the user.
2. **Ask before any destructive step.** That means formatting an SD card, erasing flash, or rotating a token. These steps are marked ⚠ below.
3. **Physical steps belong to the user.** Plugging cables, pressing BOOT/RST and inserting the card. Say exactly what to do and wait for confirmation.
4. **Verify every step** with its *Expect* check before moving on. If a check fails, use that step's *If it fails* table, then section 9. Stop after two failed attempts at the same step and report what you saw.
5. Prefer `cambot` CLI commands over raw `esptool` or `curl`. Every command accepts `--json` for machine-readable output.

Step outline: S1 tools → S2 SD card → S3 connect → S4 flash → S5 Wi-Fi → S6 verify → S7 model → S8 scene → S9 mount and aim → S10 editor (MCP).

1. What you need

Item	Notes
FORIOT ESP32-S3-CAM (OV3660, N16R8)	Or another supported board; see `cambot boards`

Item	Notes
microSD card, 8–128 GB	High-endurance recommended. 32 GB holds about 1.5 days at default settings.
USB-C **data** cable	Charge-only cables are the most common setup failure
Computer	macOS 12+, Linux, or Windows 10+, with Python 3.10+
2.4 GHz Wi-Fi	Most dual-band routers broadcast it, sometimes under a separate name
Printed case and a foot	Optional for setup; needed for mounting

S1. Install the tools

Run:

```
python3 -m pip install --user pipx && python3 -m pipx ensurepath
pipx install cambot-cli
cambot doctor
```

Expect: `doctor` ends with `doctor=ok`. It checks Python, esptool, serial drivers and ffmpeg (ffmpeg is only needed for MP4 export).

If it fails	Fix
`cambot: command not found`	Open a new terminal (pipx changed `PATH`), or run `python3 -m pipx run cambot-cli doctor`
`driver=missing ch340` (Windows, older macOS)	Install the WCH CH340/CH341 driver, then replug the board
`ffmpeg=missing`	Optional. macOS `brew install ffmpeg`; Debian/Ubuntu `sudo apt install ffmpeg`; Windows `winget install ffmpeg`

S2. Insert the SD card

User does: insert the microSD card into the slot on the **back** of the board, contacts facing the board, until it clicks. Do this with the board unplugged.

You do not need to format the card on the computer. The camera checks it at boot and reports `sd=ok`, `sd=none` or `sd=unformatted`. If it needs formatting, step S6 handles it on the camera, which avoids erasing the wrong disk on the host.

S3. Connect the TTL port

The board has two USB-C ports on its bottom edge. **Use the one labeled TTL** (a CH340 USB-serial chip). It flashes reliably and carries the logs. The other port (OTG) is the ESP32's native USB. It works, but it resets the chip every time a program opens it.

User does: plug a data cable into the **TTL** port.

Run:

```
cambot ports --json
```

Expect: exactly one entry with `"kind": "ttl"`, VID `1a86`. Typical names are `/dev/cu.usbserial-XXXX` (macOS), `/dev/ttyUSB0` (Linux) or `COM5` (Windows). Use that device as `PORT` below, or pass `--port auto`.

If it fails	Fix
No ports at all	Try another cable (charge-only cables are common), then another USB port. Avoid unpowered hubs.
Only a `"kind": "otg"` port (VID `303a`)	The cable is in the OTG port. Move it to TTL.
Linux `permission denied`	`sudo usermod -aG dialout $USER`, then log out and back in

S4. Flash the firmware ⚠

Flashing erases the previous firmware on the board. Confirm with the user first if the board was in use.

Run:

```
cambot flash --port auto --board foriot_s3cam
```

This downloads the latest release and runs esptool at **115200 baud**. This board's CH340 link drops at higher speeds. A full flash takes about two minutes.

Expect: the command ends with `flash=ok` and `boot=HELLO cambot`.

If it fails	Fix
`Failed to connect to ESP32-S3` / `No serial data received`	**User does:** hold **BOOT**, tap **RST**, release **BOOT**. Then rerun with `--no-reset`.
Fails partway through	Rerun once. Check the speed is still 115200 (`--baud 115200`) and the cable is not loose.
`psram=0` in the boot line	Wrong board profile. Rerun with the correct `--board`.

No-install alternative for people: open the web flasher at the project's GitHub Pages site in Chrome or Edge, click **Connect**, and pick the TTL port. It flashes and then asks for Wi-Fi in the same page.

S5. Join Wi-Fi

Run (the password never appears on the command line):

```
cambot provision --port auto --ssid "<SSID from user>" --tz "<IANA zone>"
# prompts for the password, or reads CAMBOT_WIFI_PASSWORD if it is set
```

Expect (JSON with `--json`):

```
wifi=ip=192.168.1.42
mdns=cambot-ab12.local
token=saved keychain:cambot/cambot-ab12
provision=ok
```

The token is stored in the OS keychain and is not printed. Later commands look it up by camera name.

If it fails (`wifi=disconnected reason=N`)	Meaning	Fix
`201`	Network not found	Wrong SSID, or a 5 GHz-only network. Ask the user for the 2.4 GHz name.
`202`, `15` or `204`	Authentication or handshake failed	Wrong password. Ask the user to re-enter it.
`203`	Association refused	MAC filtering or a full access point; the user checks the router
`provision=timeout` with no reason	Weak signal or captive portal	Move closer for setup. Guest networks with sign-in pages are not supported.

Fallback without a computer: hold **BOOT** for 5 seconds after power-on. The camera opens a Wi-Fi network `Cambot-Setup-XXXX`. Join it from a phone and use the page at `http://192.168.4.1`.

S6. Verify the camera ⚠ (SD format if needed)

Run:

```
cambot status cambot-ab12 --json
cambot snap cambot-ab12 -o cambot-test.jpg
```

Expect in `status`: `"wifi.rssi"` better than −75, `"time.synced": true`, `"sd.state": "ok"`, `"recording": true`. `cambot-test.jpg` is a sharp, correctly colored image. **Agents:** open the image and describe it to the user to confirm the camera works.

If it fails	Fix
`"sd.state": "none"`	Card not seated. Unplug the board, reseat the card, replug.
`"sd.state": "unformatted"`	⚠ With the user's OK: `cambot sd format cambot-ab12 --confirm`. This erases the card.
`"time.synced": false` after 2 minutes	The network blocks NTP. Set `cambot config set time.ntp <server>`, or use the router's address.
Image green or tinted	Normal for the first few seconds after boot; snap again
Image black or `camera=fail`	Unplug. Reseat the camera flex cable (contacts toward the board) and the module.
`mdns` name does not resolve	Use the IP from S5. Windows without Bonjour, and some routers, block mDNS.

S7. Connect a model (optional)

Recording, live view and clips work without a model. The semantic index and `ask` need one. Any OpenAI-compatible endpoint works, including a model server on your own network.

Run (the key is read from stdin, never from an argument):

```
cambot config set cambot-ab12 model.base_url "<base URL>" model.name "<vision model>"
printf '%s' "$CAMBOT_MODEL_KEY" | cambot config set cambot-ab12 model.api_key -
cambot config set cambot-ab12 budget.frames_per_day 2000
cambot ask cambot-ab12 "What do you see right now?"
```

Expect: a one-paragraph description of the scene, and `model=ok` in `cambot status`.

If it fails	Fix
`model=auth`	Wrong key. The user re-enters it.
`model=no_vision`	The chosen model does not accept images. Pick one listed in `docs/models.md`.
`model=tls`	Private server with its own CA: `cambot config set cambot-ab12 model.ca_pem @ca.pem`

S8. Describe the scene and what to watch for

The scene profile and watch list are what make search accurate. Write them with the user.

```
cambot profile set cambot-ab12 --file scene.yaml
```

```
# scene.yaml
profile: >
  Overhead view of an Opentrons OT-2 liquid handler. Tip racks are in slots
  1-3 on the left, a 96-well plate is in slot 5, and the trash is top right.
watch:
  - tip pickup
  - tip eject
  - liquid dispense into plate
  - person or hand in frame
  - error light on
index:
  active_s: 3
  idle_s: 60
```

Expect: `profile=ok`. After a few minutes of activity, `cambot events cambot-ab12 --since -10m` lists events.

Robot marks (recommended): have the robot's controller call the camera whenever it starts a step. This gives exact, searchable labels:

```
curl -s -X POST http://cambot-ab12.local/api/v1/marks \
  -H "Authorization: Bearer $CAMBOT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"label":"step 12: pick tip"}'
```

S9. Mount and aim

1. Choose a foot: the **screw clamp** for edges up to 35 mm, the **magnet puck** for steel, or any GoPro-style mount.
2. Join the foot to the camera's knuckle with the M5 bolt, leaving it loose.
3. Open the live view at `http://cambot-ab12.local/` (or run `cambot live cambot-ab12`) and aim the camera. Then tighten the bolt.

Placement rules:

- Frame the work area tightly. Keep the robot's key action, such as the tip rack, in the middle third of the image.
- Keep bright windows and lamps out of the frame; backlight ruins exposure.
- Point the antenna end (top of the case) away from metal, and keep it at least 10 mm clear of steel surfaces. Check `"wifi.rssi"` after mounting.

- Do not cover the side vents.

Agents: fetch a snapshot after mounting and confirm the watched objects are visible and in focus.

S10. Add the camera to your editor (MCP)

Run:

```
cambot mcp-config cambot-ab12
```

It prints an entry for `.cursor/mcp.json` (project) or `~/.cursor/mcp.json` (global):

```
{
  "mcpServers": {
    "cambot-ab12": {
      "url": "http://cambot-ab12.local/mcp",
      "headers": { "Authorization": "Bearer ${env:CAMBOT_TOKEN}" }
    }
  }
}
```

The token is referenced through an environment variable, so it is never written into the file. Export `CAMBOT_TOKEN` from the keychain with `cambot token export cambot-ab12` in the shell that launches the editor.

Expect: the editor lists tools such as `search_index`, `inspect_window` and `make_clip`. Test it by asking the agent: *"Using cambot-ab12, show me the last time something moved."*

9. Troubleshooting reference

Symptom	Likely cause	Fix
Board resets whenever a serial monitor opens	Monitor attached to the OTG port	Use the TTL port for logs
Upload fails at 921600	CH340 limit on this board	Use 115200 (the CLI default)
Random reboots with SD inserted	SD driver started with default pins, which overlap PSRAM	Firmware bug; report it with `cambot logs` output
`recording=false`, `sd.full=true`	Eviction disabled or the card is too small	`cambot config set storage.reserve_pct 10`
Index stopped, recording fine	Daily budget reached	Shown in `status.budget`; raise it or wait for the reset
Clips stutter in the browser	Wi-Fi throughput	Use `cambot pull` / `export`, or lower `capture.fps`

Symptom	Likely cause	Fix
Lost token		⚠ `cambot token rotate cambot-ab12 --port auto` over USB

Appendix A. Log keys

The TTL log at 115200 uses one `key=value` record per line, so it can be grepped:

Key	Values
`boot`	`HELLO cambot`, firmware version, board
`camera`	`ok pid=0x3660`, `fail err=...`
`sd`	`ok size_gb=29.7`, `none`, `unformatted`
`wifi`	`connecting`, `ip=...`, `disconnected reason=N`
`mdns`	`cambot-XXXX.local`
`time`	`synced`, `unsynced`
`rec`	`segment=... frames=300 dropped=0`
`index`	`keyframes=... budget_left=...`
`model`	`ok`, `auth`, `tls`, `no_vision`, `rate_limited`